Relatório Comparativo SonarQube: Fase 02 x Fase 03

1. Resultados

Métrica	Fase 02	Fase 03	Variação
Code Smells	57	75	+18 (31,6%)
Bugs	0	0	=
Vulnerabilities	0	0	=
Coverage	30,7%	63,7%	+33 p.p.
Duplications	0%	0%	=
Lines of Code	639	1468	+829 (129,7%)
Quality Gate	Passed	Passed	=
Testes executados	7	39	+32
Testes aprovados	6	39	+33
Testes reprovados	1	0	-1

2. Análise por métrica

2.1 Métricas que melhoraram

Coverage (30,7% para 63,7%) - Aumento de 33 pontos percentuais, graças a 32 testes novos com Mockito e da refatoração para testabilidade. Considerando apenas as classes de regra (model, enums, service, repository), o relatório JaCoCo do repositório (coverage/html) mostra cobertura acima das metas da fase (80% de linhas e 70% de branches). O valor global do Sonar é menor porque inclui o pacote view, que contém apenas entrada e saída de console, não possui testes específicos e está fora da meta definida pelo enunciado para classes de regra.

Testes (6 de 7 para 39 de 39) - O teste reprovado na Fase 02 (testUsoLimiteDeItensExcedido) revelou o BUG-01 (a Mochila aceitava o terceiro uso de item sem lançar exceção), corrigido na refatoração. Não houve testes reprovados na Fase 03.

Bugs, Vulnerabilities e Duplications - Permaneceram em 0 nas duas fases, mesmo com o crescimento de 130% no código. O Quality Gate foi aprovado nas duas análises.

2.2 Métrica que piorou: Code Smells

Composição dos 75 smells na Fase 03:

Regra do Sonar	**Quantidade**	**% do total**
Substituir System.out por logger	68	90,7%
Construtor com 10 parâmetros (máximo 7)	1	1,3%
Método com 8 parâmetros (máximo 7)	1	1,3%
Mais de um break/continue em um laço	1	1,3%
Usar import estático (mock, when, times, verify)	4	5,3%

Justificativas:

- System.out (68 ocorrências). É a regra que mais apareceu nas duas fases e não representa piora estrutural. O sistema é uma aplicação de console, e a interação com o usuário por System.out é intencional. Na Fase 03, os SysOuts foram isolados no pacote view (ConsoleRegistro, ConsoleEscolha, ConsoleNotificador, ConsoleAvisoTorneio, Main), enquanto as classes de regra não imprimem mais nada, para não violar a Inversão de Dependência. O aumento da quantidade vem do crescimento de classes de apresentação (torneio, conquistas, registro de novos eventos como precisão e contra-ataque), não de mais código mal estruturado.
- Construtor com 10 parâmetros (BatalhaService). Consequência direta do Princípio da Inversão de Dependência: todos os colaboradores (gerador aleatório, calculadora, iniciativa, escolhas de ação, registro, repositório e serviço de terreno) são recebidos pelo construtor.

3. Conclusão

A Fase 03 aumentou o código do projeto em 130% com a adição de torneio, repositório de resultados, registro de eventos, conquistas, serviço de terreno por horário, precisão e contra-ataque, mantendo Bugs, Vulnerabilities e Duplications em zero e o Quality Gate aprovado. O Coverage subiu de 30,7% para 63,7% (excluindo a view que não tem testes, a cobertura passa de 80%). A única métrica que piorou em valor absoluto foi o Code Smells, porém teve crescimento proporcionalmente menor que o do código, logo, também pode ser considerado uma melhora em relação à fase 02.